

Árvore AVL Aumentada sob Cargas SOSD

Amanda Souza (2510101045), Milena Silva (2510100191), Rafaela Nunes (2510101040)

¹Universidade Federal do Pampa

{amandadds, milenacs2, rafaelanunes}.aluno@unipampa.edu.br

Vídeo da apresentação

<https://drive.google.com/file/d/1RuJahYzTP7xFd5BAIHLhYXINWvLnhIYy/view?usp=sharing>

1. Introdução

Árvores representam relações hierárquicas por meio de nós e arestas. Um nó pode ter descendentes, e o percurso entre esses elementos define caminhos, níveis e subárvores. Essa organização aparece em índices, sistemas de arquivos, compiladores e outras aplicações que precisam localizar ou organizar dados sem percorrer uma coleção inteira.

Uma árvore binária restringe cada nó a, no máximo, dois filhos. Na árvore binária de busca (BST), as chaves menores que a chave de um nó ficam à esquerda, enquanto as maiores ficam à direita. Busca, inserção e remoção percorrem um caminho da raiz até uma posição da árvore. O custo dessas operações depende da altura. Uma BST com distribuição equilibrada tem caminhos curtos, mas inserções ordenadas podem produzir uma cadeia com altura linear, ou seja, onde cada nó tem apenas um filho e a estrutura da árvore acaba sendo semelhante a uma lista.

A árvore AVL evita essa estrutura semelhante a uma lista ao limitar a diferença entre as alturas das subárvores de cada nó. Quando uma inserção ou remoção viola esse limite, rotações locais restauram o balanceamento. Uma característica interessante é que árvores podem armazenar metadados derivados de cada subárvore. Por exemplo, tamanho, altura e valor máximo permitem responder consultas de ordem e agregação sem visitar todos os nós.

Mesmo que a inserção de metadados e rotações mantenham a estrutura de árvore balanceada, elas introduzem custos adicionais para cada operação. Por exemplo, durante uma rotação, pode ser necessário atualizar o tamanho e o valor máximo de cada nó afetado, o que gera um custo computacional adicional. Por outro lado, uma árvore com balanceamento adequado pode reduzir significativamente o número de nós visitados durante uma busca, resultando em um desempenho melhor em operações de consulta quando comparada com uma lista ordenada.

Este trabalho visa avaliar o desempenho de operações de inserção, remoção e busca em uma árvore AVL aumentada com metadados de tamanho e valor máximo, em comparação com uma lista ordenada. A avaliação considera diferentes escalas, ordens de inserção e enviesamentos de acesso, medindo latências e analisando como essas variáveis afetam o desempenho relativo das duas estruturas.

O restante deste trabalho está organizado como segue. A Seção 2 apresenta as decisões de design e a organização da implementação. A Seção 3 descreve a metodologia experimental e discute os resultados. A Seção 4 sintetiza as conclusões e registra o uso de inteligência artificial.

2. Implementação

2.1. Decisões de Design

A AVL foi escolhida por fornecer altura logarítmica com uma regra local de balanceamento. Uma BST sem balanceamento teria implementação menor, mas a ordem `sorted` poderia produzir altura linear. Uma árvore rubro-negra ou uma treap também atenderiam ao limite assintótico, embora introduzissem, respectivamente, regras adicionais de cor ou uma fonte extra de aleatoriedade.

Cada nó mantém a chave, os dois filhos, o pai, a altura, o tamanho e o máximo da subárvore. O ponteiro `parent` permite subir até a raiz sem recursão ou pilha externa. Seu custo é uma referência por nó e a obrigação de atualizar os dois sentidos de cada ligação durante remoções e rotações. O método `refresh` recalcula altura, tamanho e máximo a partir dos filhos.

Os invariantes são a ordenação estrita de BST, a correspondência entre filhos e pais, a raiz sem pai, o fator de balanceamento entre -1 e 1 e a correção dos três metadados. Nas rotações, o nó que desce é atualizado antes do pivô que sobe. Essa ordem evita que o novo topo use metadados antigos. `rank` e `select` descartam subárvores completas pelo campo de tamanho. Como o agregado do grupo é a própria chave, `range_agg` procura o predecessor do limite superior e verifica o limite inferior.

Na rotação à esquerda, o pivô é o filho direito. A subárvore esquerda do pivô passa ao lado direito do nó rebaixado. Suas chaves estão entre as duas chaves locais, o que preserva a ordenação da BST.

A rotação à direita aplica o argumento simétrico. `replace_subtree` liga o pivô ao antigo pai ou à raiz. Cada ponteiro de filho alterado recebe o `parent` correspondente.

O nó rebaixado é atualizado antes do pivô. Assim, altura, tamanho e máximo são calculados a partir de filhos já corretos. Casos duplos compõem duas rotações simples e preservam os mesmos invariantes.

Após cada mutação, `rebalance_from` percorre os ancestrais. Ele recalcula o fator e aplica a rotação adequada quando a diferença de alturas sai de $[-1, 1]$.

Os testes exercitam LL, RR, LR e RL. Outro teste executa 1.000 mutações com semente 5, compara as consultas com referências independentes e chama `assert_valid` após cada operação.

O *baseline* usa bisseção sobre uma lista Python. Busca e localização da posição acabam sendo eficientes por se beneficiarem da estrutura da lista, mas inserções e remoções internas deslocam elementos subsequentes, o que requer mais operações. Por fim, inserções ordenadas são um caso favorável, pois ocorrem ao final da lista.

2.2. Organização e Implementação

O código está separado por responsabilidade. `src/augmented_avl.py` contém `Node` e `AugmentedAVLTree`; `src/run_trace.py` executa rastros e compara buscas com o oráculo; `src/benchmark.py` implementa a lista ordenada e coleta métricas; `src/run_experiments.py` expande a matriz JSON, cria identificadores estáveis e persiste checkpoints; `src/plot_results.py` agrega os pares e produz as figuras.

Testes ficam em `tests/`, configurações e material didático em `docs/`, resultados em `results/` e o manuscrito em `report/`.

A implementação usa classes para encapsular as duas estruturas, `dataclass` com `slots` para os nós, anotações de tipo e iteradores para processar rastros sem carregá-los integralmente. `array("Q")` armazena as latências com oito bytes por amostra. `pathlib`, gerenciadores de contexto, `csv`, `hashlib` e `itertools.product` sustentam a execução reproduzível. O `uv`¹ fixa e instala as dependências.

O executor produz um `run_id` determinístico para cada combinação e grava cada resultado. Uma nova execução ignora somente casos concluídos com sucesso, o que permite retomar a bateria sem repetir medições válidas. A correção é verificada por testes feitos com o método `assert_valid` e pela comparação das buscas com o arquivo `.expected`. A versão medida passou em 28 testes e verificações com `Ruff`² e `mypy`³.

3. Experimentos

3.1. Descrição dos Experimentos

Os experimentos respondem a duas perguntas: (Q1) como escala e ordem de inserção afetam o desempenho relativo da AVL e da lista ordenada? (Q2) como o enviesamento Zipfiano afeta a remoção? Q1 é examinada pela variação de escala e pela Figura 1. Q2 é examinada pela variação de θ e pela Figura 2.

Tabela 1. Ambiente definido para a bateria definitiva.

Componente	Especificação definitiva
Dispositivo	IdeaPad 3 15ALC6 (82MF)
Sistema operacional	Ubuntu 26.04 LTS, kernel Linux 7.0.0-27-generic
Processador	AMD Ryzen 5 5500U, 12 threads, 2,10 GHz
Memória	8 GB DDR4, 3.200 MHz
Armazenamento	SSD M.2 NVMe de 256 GB
Runtime	CPython 3.13.5 (Clang 20.1.4)

O conjunto `books_200M_uint32` fornece chaves de 32 bits. O estudo de escala usa 1 mil, 10 mil, 100 mil e 1 milhão de operações, com universo e carga máxima iguais a cada escala. Mantivemos a mistura 45 : 25 : 30, reserva de 10%, 70% de acertos em buscas, 5% de remoções ausentes, semente 5 e $\theta = 0,99$. As ordens `sorted` e `shuffle` e as duas implementações formam os pares comparados. O estudo de sensibilidade fixa 100 mil operações e varia θ em $\{0; 0,6; 0,99; 1,2\}$. Cada combinação possui cinco repetições, totalizando 140 execuções e 28,22 milhões de operações medidas.

A matriz contém quatro escalas decimais, de 10^3 a 10^6 operações. Os extremos diferem por um fator de 10^3 . Nenhuma execução com 10^7 operações foi realizada, por isso as conclusões ficam restritas à faixa medida.

O cronômetro `perf_counter_ns` envolve somente a chamada da estrutura para cada operação. Para inserção, remoção e busca, registramos média, p50 e p99. O tempo

¹<https://github.com/astral-sh/uv>

²<https://github.com/astral-sh/ruff>

³<https://github.com/python/mypy>

total do rastro inclui leitura e despacho. Também registramos rotações, tamanho e altura finais. As razões dos gráficos são calculadas entre lista e AVL no mesmo rastro e repetição; a linha conecta a mediana das cinco razões. Os pontos esmaecidos mostram as repetições individuais.

A reprodução usa `docs/estudo-empirico-grupo-5-relatorio.json` e os comandos documentados no `README.md`. O CSV preserva parâmetros, ambiente e identificadores. Rastros completos são reutilizados entre implementações, enquanto resultados parciais permitem retomada. O material preparado para publicação no repositório GitHub inclui código, `uv.lock`, configurações, instruções, resultados, gráficos e os registros de interação com inteligência artificial.

3.2. Resultados e Discussão

As Tabelas 2 e 3 apresentam as medianas das cinco repetições. Cada célula usa somente execuções com $\theta = 0,99$ e informa latência em μs .

Tabela 2. Latências por escala com inserções ordenadas (em μs).

Escala	Estrutura	Inserção			Remoção			Busca		
		Média	p50	p99	Média	p50	p99	Média	p50	p99
10^3	AVL	9,809	9,987	21,860	7,621	7,613	17,879	2,540	2,444	3,352
10^3	Lista	2,725	2,863	3,772	2,908	2,864	3,841	2,786	2,863	3,772
10^4	AVL	12,750	12,782	24,095	9,816	10,057	20,115	2,803	2,863	3,423
10^4	Lista	3,567	3,702	4,400	3,807	3,841	5,168	3,597	3,771	4,680
10^5	AVL	15,888	15,644	27,657	12,821	13,200	24,585	3,288	3,283	4,819
10^5	Lista	4,360	4,330	5,727	5,263	4,819	9,429	4,534	4,400	6,216
10^6	AVL	19,764	18,299	31,010	16,954	17,041	30,451	4,276	3,911	7,054
10^6	Lista	5,152	5,168	6,775	12,633	6,774	55,175	5,860	5,727	8,242

Tabela 3. Latências por escala com inserções embaralhadas (em μs).

Escala	Estrutura	Inserção			Remoção			Busca		
		Média	p50	p99	Média	p50	p99	Média	p50	p99
10^3	AVL	8,981	8,590	14,318	7,622	7,613	11,873	2,522	2,375	3,352
10^3	Lista	2,891	2,864	3,841	2,974	2,864	3,841	2,847	2,863	3,423
10^4	AVL	11,710	11,035	22,280	9,872	9,987	19,975	2,816	2,863	3,771
10^4	Lista	3,930	3,841	5,657	3,915	3,841	5,308	3,637	3,771	4,889
10^5	AVL	14,922	14,178	26,680	12,739	12,990	24,305	3,304	3,283	4,400
10^5	Lista	6,471	6,076	12,362	5,609	5,308	9,079	4,494	4,400	5,727
10^6	AVL	20,956	18,997	33,454	17,033	17,251	30,800	4,437	4,330	7,054
10^6	Lista	26,137	21,093	77,246	18,919	14,806	55,734	6,103	6,146	8,870

Na configuração principal, com 1 milhão de operações e inserções ordenadas, a lista apresentou menor média de inserção. Na remoção, sua média também foi menor, mas seu p99 foi 1,81 vez o p99 da AVL.

A AVL apresentou menor latência de busca nessa configuração. A lista acrescenta chaves crescentes ao final, enquanto a AVL percorre ancestrais, atualiza metadados e pode executar rotações.

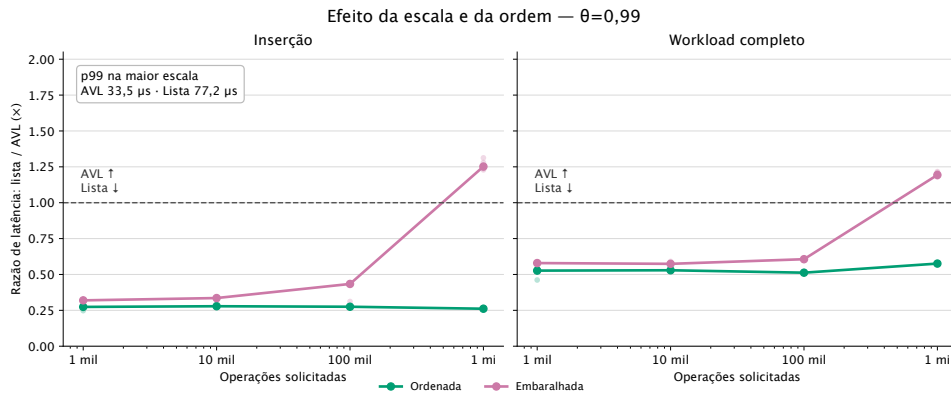


Figura 1. Razão de latência lista/AVL para $\theta = 0,99$. Valores acima de 1 favorecem a AVL, enquanto valores abaixo de 1 favorecem a lista.

Na Figura 1, a razão de inserção sob *sorted* permaneceu próxima de 0,27. Ela passou de 0,274 em 1 mil operações para 0,261 em 1 milhão. Na maior escala, a razão do workload completo foi 0,576, o que corresponde a uma lista 1,74 vezes mais rápida. Inserções crescentes são acrescentadas ao final da lista, enquanto a AVL percorre ancestrais, atualiza metadados e pode executar rotações.

Sob *shuffle*, a razão do workload completo passou de 0,579 em 1 mil operações para 0,606 em 100 mil e 1,193 em 1 milhão. O ponto de cruzamento observado ficou entre 100 mil e 1 milhão de operações. Na maior escala, a AVL foi 1,19 vezes mais rápida. A razão de inserção chegou a 1,252, e o p99 foi 33,454 μs na AVL e 77,246 μs na lista, uma razão de 2,30. Inserções internas deslocam uma parcela crescente do vetor. A AVL terminou com altura 21 para 212.376 chaves no caso embaralhado. Sob *sorted*, a altura final foi 19 após 496.063 rotações.

Na AVL, inserção, remoção e busca percorrem caminhos de $O(\log n)$. Sob *sorted*, o tamanho final passou de 259 para 212.648 chaves, e seu logaritmo na base 2 cresceu 2,21 vezes.

No mesmo intervalo, as médias da AVL cresceram 2,02 vezes na inserção, 2,23 na remoção e 1,68 na busca. Sob *shuffle*, os fatores foram 2,33, 2,24 e 1,76, enquanto o logaritmo cresceu 2,26 vezes.

Esses fatores são compatíveis com caminhos logarítmicos, mas não demonstram sozinhos o limite assintótico. Alocação, ponteiros, atualização dos metadados e rotações também compõem a latência da AVL.

Na lista, busca e localização da posição usam bisseção em $O(\log n)$. Inserções internas e remoções deslocam elementos em $O(n)$. Sob *sorted*, a inserção ocorre no fim, mas ainda executa a bisseção.

$O(n)$ significa que o trabalho pode crescer na mesma proporção que a quantidade de elementos. Se a lista dobrar, uma operação no início pode precisar deslocar aproximadamente o dobro de posições.

A bisseção encontra a posição sem percorrer a lista inteira. Porém, depois dessa busca, inserir ou remover no meio ainda exige mover todos os elementos posteriores. Esse

deslocamento domina o custo da operação.

Sob `shuffle`, a média e o p99 da inserção da lista cresceram 9,04 e 20,11 vezes. O p99 da remoção cresceu 14,51 vezes. A média da busca cresceu 2,14 vezes.

Os deslocamentos usam memória contígua e rotinas nativas, enquanto o tamanho varia durante o rastro. Por isso, o crescimento medido não precisa igualar o fator da escala, embora o custo linear continue explicando a tendência.

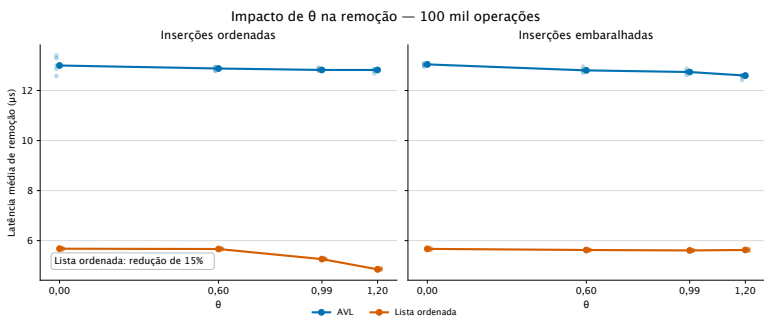


Figura 2. Efeito de θ na remoção em 100 mil operações, que representa a maior escala com variação completa dentre os experimentos realizados.

Na Figura 2, a latência média de remoção da lista sob `sorted` caiu de 5,678 para 4,854 μs entre $\theta = 0$ e $\theta = 1,2$, uma redução de 15%. A AVL permaneceu entre 12,821 e 12,999 μs . Com inserções embaralhadas, ambas variaram pouco: a AVL ficou entre 12,597 e 13,043 μs , e a lista entre 5,609 e 5,671 μs .

Os rastros de cada ordem mantiveram as mesmas contagens de inserções e remoções entre os valores de θ . A Tabela 4 compara as rotações simples executadas pela AVL.

Tabela 4. Rotações medianas da AVL em 100 mil operações.

θ	sorted		shuffle	
	Rotações	por 1.000 mutações	Rotações	por 1.000 mutações
0	49.970	713,1	38.346	546,6
0,6	49.756	710,1	38.123	543,4
0,99	49.613	708,0	35.541	506,6
1,2	48.402	690,8	32.309	460,6

Entre $\theta = 0$ e $\theta = 1,2$, as rotações por mil mutações caíram 3,1% sob `sorted` e 15,7% sob `shuffle`. As alturas finais medianas permaneceram em 16 e 17, respectivamente.

A associação mostra que a concentração alterou a atividade de rebalanceamento nesses rastros. Ela não demonstra que valores maiores de θ reduzam rotações em outras cargas.

O parâmetro θ atua sobre posições do conjunto vivo. Após uma remoção, o gerador ocupa a posição com a última chave do conjunto. Assim, uma posição quente não permanece ligada ao mesmo valor.

A repetição de posições pode favorecer reutilização de dados, mas a disposição dos nós Python e a troca de chaves dificultam atribuir a latência ao cache. Sem contadores de hardware, localidade permanece uma hipótese, não uma conclusão.

4. Considerações Finais

4.1. Visão Geral

Uma estrutura de dados combina uma garantia teórica com os custos concretos de sua representação. A altura logarítmica protege a AVL de caminhos longos, mas ponteiros, metadados e rotações têm custo. A lista ordenada dispensa esse trabalho quando as chaves chegam em ordem crescente; o mesmo arranjo se torna caro quando as inserções alcançam posições internas.

Na maior escala, a lista foi 1,74 vezes mais rápida no workload ordenado, enquanto a AVL foi 1,19 vezes mais rápida no workload embaralhado. O p99 da inserção embaralhada da lista foi 2,30 vezes o p99 da AVL. A variação de θ reduziu em 15% a latência média de remoção da lista com inserções ordenadas. A AVL variou pouco. No workload embaralhado, o ponto de cruzamento observado ficou entre 100 mil e 1 milhão de operações.

Os resultados descrevem uma máquina, uma implementação e a faixa de 10^3 a 10^6 operações. Os quatro pontos observados localizam o cruzamento do workload embaralhado entre 100 mil e 1 milhão.

Os extremos da faixa diferem por um fator de 10^3 , e nenhuma execução com 10^7 operações foi realizada. Portanto, o estudo não sustenta extrapolações além de 1 milhão de operações.

Cinco repetições não representam variações entre máquinas. Uma replicação em outro processador permitiria avaliar a estabilidade do cruzamento. Contadores de hardware permitiriam testar a hipótese de localidade de cache.

4.2. Uso de Inteligência Artificial

O grupo adotou transparência como critério para o uso de inteligência artificial.

O diretório `conversation-logs` reúne os prompts enviados e as respostas usadas durante o projeto. Esse material pode ser lido no GitHub⁴.

A skill `decision-grill`, extensão local de `grill-with-docs`⁵, apoiou o registro das decisões iniciais. O fluxo Writing Assistant organizou as perguntas experimentais e revisou a clareza do texto.

O apoio foi realizado com OpenAI Codex v0.144.3 e os modelos GPT 5.6 Luna e GPT 5.6 Sol, ambos configurados com esforço alto. As ferramentas auxiliaram o planejamento, a implementação, os testes, a análise e a redação.

Os programas do projeto geraram rastros, medições e gráficos. O grupo conferiu os resultados nos dados, revisou os artefatos e permanece responsável pelas decisões e conclusões.

⁴[Acessar os prompts e as respostas](#)

⁵[Acessar a skill grill-with-docs](#)